04. 타임아웃 관리 시스템

목차

1. 타임아웃 관리자 개념
2. 적응형 타임아웃 알고리즘
3. 네트워크 지연시간 추적
4. 백오프 전략
5. 타임아웃 통계 및 모니터링
6. 성능 최적화

타임아웃 관리자 개념

`TimeoutManager` 는 네트워크 조건에 따라 동적으로 타임아웃을 조정하는 핵심 컴포넌트입니다. 네트워크 지연시간 측정값을 기반으로 타임아웃을 적응적으로 조정하여 불필요한 재전송을 줄이고 성능을 최적화합니다.

기본 구조

```
pub struct TimeoutManager {
base_timeout: Duration, // 기본 타임아웃
current_timeout: Duration, // 현재 타임아웃
network_latency_history: VecDeque<Duration>, // 지연시간 히스토리
adaptation_factor: f64, // 적응 계수
```

```
max_timeout: Duration, // 최대 타임아웃
min_timeout: Duration, // 최소 타임아웃
history_size: usize, // 히스토리 크기
consecutive_timeouts: u32, // 연속 타임아웃 횟수
last_successful_round: Option<Instant>, // 마지막 성공 시간
exponential_backoff_factor: f64, // 지수 백오프 계수
linear_decrease_amount: Duration, // 선형 감소량
}
```

타임아웃 관리자 생성

```
impl TimeoutManager {
pub fn new(
base_timeout: Duration,
adaptation_factor: f64,
max_timeout: Duration,
min_timeout: Duration,
history_size: usize,
) -> Self {
Self {
base_timeout,
current_timeout: base_timeout,
network_latency_history: VecDeque::with_capacity(history_size),
```

```
adaptation_factor,
max_timeout,
min_timeout,
history_size,
consecutive_timeouts: 0,
last_successful_round: None,
exponential_backoff_factor: 1.5,
linear_decrease_amount: Duration::from_millis(100),
}
}
}
```

커스텀 백오프 매개변수와 함께 생성

```
impl TimeoutManager {
pub fn with_backoff_params(
base_timeout: Duration,
adaptation_factor: f64,
max_timeout: Duration,
min_timeout: Duration,
history_size: usize,
exponential_backoff_factor: f64,
linear_decrease_amount: Duration,
```

```
) -> Self {
Self {
base_timeout,
current_timeout: base_timeout,
network_latency_history: VecDeque::with_capacity(history_size),
adaptation_factor,
max_timeout,
min_timeout,
history_size,
consecutive_timeouts: 0,
last_successful_round: None,
exponential_backoff_factor,
linear_decrease_amount,
}
}
}
```

적응형 타임아웃 알고리즘

네트워크 지연시간 기록 및 타임아웃 업데이트

```
impl TimeoutManager {
pub fn record_latency(&mut self, latency: Duration) {
```

```
// 히스토리 크기 제한
if self.network_latency_history.len() >= self.history_size {
self.network_latency_history.pop_front();
}
self.network_latency_history.push_back(latency);
// 타임아웃 업데이트
self.update_timeout();
}
fn update_timeout(&mut self) {
if self.network_latency_history.is_empty() {
return;
}
// 최근 지연시간의 이동 평균 계산
let total_latency: Duration = self.network_latency_history.iter().sum();
let avg_latency = total_latency / self.network_latency_history.len() as u32;
// 더 나은 적응을 위한 분산 계산
let variance = self.calculate_latency_variance(avg_latency);
let variance_factor = 1.0 + (variance.as_millis() as f64 / self.base_timeout.as
// 평균 지연시간과 분산을 기반으로 타임아웃 적응
let latency_ratio = avg_latency.as_millis() as f64 / self.base_timeout.as_milli
let adaptation_multiplier = 1.0 + self.adaptation_factor * latency_ratio * vari
let adapted_timeout = Duration::from_millis(
(self.base_timeout.as_millis() as f64 * adaptation_multiplier) as u64
```

```
);
// 최소/최대 경계로 제한
self.current_timeout = adapted_timeout.clamp(self.min_timeout, self.max_timeout)
}
}
```

지연시간 분산 계산

```
impl TimeoutManager {
fn calculate_latency_variance(&self, avg_latency: Duration) -> Duration {
if self.network_latency_history.len() < 2 {
return Duration::from_millis(0);
}
let variance_sum: u128 = self.network_latency_history
.iter()
.map(|&latency| {
let diff = latency.as_millis() as i128 - avg_latency.as_millis() as i128;
(diff * diff) as u128
})
.sum();
let variance = variance_sum / self.network_latency_history.len() as u128;
Duration::from_millis((variance as f64).sqrt() as u64)
}
```

```
}
```

네트워크 지연시간 추적

평균 지연시간 계산

```
impl TimeoutManager {
fn calculate_average_latency(&self) -> Option<Duration> {
if self.network_latency_history.is_empty() {
None
} else {
let total: Duration = self.network_latency_history.iter().sum();
Some(total / self.network_latency_history.len() as u32)
}
}
}
```

타임아웃 조정 필요성 판단

```
impl TimeoutManager {
pub fn should_adjust_timeout(&self) -> bool {
// 충분한 샘플이 있거나 최근 타임아웃이 있는 경우 조정
self.network_latency_history.len() >= 3 || self.consecutive_timeouts > 0
```

```
}

}
```

백오프 전략

지수 백오프 (타임아웃 증가)

```
impl TimeoutManager {

pub fn increase_timeout(&mut self) {

self.consecutive_timeouts += 1;

// 연속 타임아웃 고려한 지수 백오프 적용

let backoff_multiplier = self.exponential_backoff_factor.powf(self.consecutive_

let new_timeout = Duration::from_millis(

(self.current_timeout.as_millis() as f64 * backoff_multiplier) as u64

);

self.current_timeout = new_timeout.min(self.max_timeout);

}

}
```

선형 감소 (타임아웃 감소)

```
impl TimeoutManager {

pub fn decrease_timeout(&mut self) {
```

```
// 성공적인 작업 시 연속 타임아웃 리셋
self.consecutive_timeouts = 0;
self.last_successful_round = Some(Instant::now());
// 선형 감소 적용
self.current_timeout = self.current_timeout
.saturating_sub(self.linear_decrease_amount)
.max(self.min_timeout);
}
}
```

타임아웃 및 성공 이벤트 기록

```
impl TimeoutManager {
pub fn record_timeout(&mut self) {
self.increase_timeout();
}
pub fn record_success(&mut self) {
self.decrease_timeout();
}
}
```

타임아웃 리셋

```
impl TimeoutManager {
pub fn reset_timeout(&mut self) {
self.current_timeout = self.base_timeout;
self.network_latency_history.clear();
self.consecutive_timeouts = 0;
self.last_successful_round = None;
}
}
```

타임아웃 통계 및 모니터링

타임아웃 통계 구조

```
#[derive(Debug, Clone)]
pub struct TimeoutStats {
pub current_timeout: Duration,
pub base_timeout: Duration,
pub consecutive_timeouts: u32,
pub avg_latency: Option<Duration>,
pub latency_samples: usize,
}
```

통계 수집

```rust
impl TimeoutManager {
pub fn get_stats(&self) -> TimeoutStats {
TimeoutStats {
current_timeout: self.current_timeout,
base_timeout: self.base_timeout,
consecutive_timeouts: self.consecutive_timeouts,
avg_latency: self.calculate_average_latency(),
latency_samples: self.network_latency_history.len(),
}
}
}
```

현재 타임아웃 조회

```rust
impl TimeoutManager {
pub fn current_timeout(&self) -> Duration {
self.current_timeout
}
}
```

성능 최적화

Fast Path에서의 타임아웃 관리 통합

```
impl FastPathConsensus {
pub fn record_network_latency(&mut self, latency: Duration) {
self.timeout_manager.record_latency(latency);
self.performance_metrics.record_latency(latency);
}
pub fn record_round_success(&mut self, round: Round) {
self.timeout_manager.record_success();
self.performance_metrics.record_success(round);
}
pub fn get_timeout_stats(&self) -> TimeoutStats {
self.timeout_manager.get_stats()
}
}
```

AutobahnController에서의 타임아웃 관리

```
impl AutobahnController {
pub async fn record_network_latency(&self, latency: Duration) -> Result<(), A
let mut timeout_manager = self.timeout_manager.write().await;
timeout_manager.record_latency(latency);
Ok(())
}
pub async fn get_timeout_stats(&self) -> TimeoutStats {
```

```
let timeout_manager = self.timeout_manager.read().await;
timeout_manager.get_stats()
}
}
```

네트워크 조건 기반 타임아웃 조정

```
impl AutobahnController {
async fn should_fallback_to_slow_path(&self, metrics: &AutobahnMetrics, timeou
// 연속 타임아웃 확인
if timeout_stats.consecutive_timeouts >= 3 {
return Ok(true);
}
// 네트워크 조건 확인
if let Some(avg_latency) = timeout_stats.avg_latency {
if avg_latency > timeout_stats.current_timeout / 2 {
return Ok(true);
}
}
Ok(false)
}
```

```
}
```

타임아웃 최적화 전략

네트워크 조건별 타임아웃 설정

저지연 네트워크 (<10ms)

```
let timeout_manager = TimeoutManager::new(
Duration::from_millis(25), // base_timeout
0.25, // adaptation_factor (공격적)
Duration::from_millis(500), // max_timeout
Duration::from_millis(10), // min_timeout
25, // history_size
);
```

고지연 네트워크 (>100ms)

```
let timeout_manager = TimeoutManager::new(
Duration::from_millis(300), // base_timeout
0.05, // adaptation_factor (보수적)
Duration::from_millis(10000), // max_timeout
Duration::from_millis(100), // min_timeout
200, // history_size
```

```
);
```

불안정한 네트워크

```
let timeout_manager = TimeoutManager::with_backoff_params(
Duration::from_millis(200), // base_timeout
0.1, // adaptation_factor
Duration::from_millis(15000), // max_timeout
Duration::from_millis(50), // min_timeout
100, // history_size
2.0, // exponential_backoff_factor
Duration::from_millis(25), // linear_decrease_amount
);
```

동적 타임아웃 조정 예제

```
impl TimeoutManager {
pub fn adaptive_timeout_example(&mut self) {
// 네트워크 스트레스 시뮬레이션
let stress_latencies = [100, 200, 400, 800, 1600];
for &latency in &stress_latencies {
self.record_latency(Duration::from_millis(latency));
}
println!("Stress timeout: {:?}", self.current_timeout());
```

```
// 네트워크 개선 시뮬레이션
for _ in 0..5 {
self.record_latency(Duration::from_millis(50));
self.record_success();
}
println!("Improved timeout: {:?}", self.current_timeout());
}
}
```

타임아웃 관리 모범 사례

1. 점진적 조정

- 급격한 타임아웃 변경 방지
- 네트워크 조건 변화에 점진적 적응

2. 경계 설정

- 최소/최대 타임아웃으로 극단적 값 방지
- 애플리케이션 요구사항에 맞는 경계 설정

3. 히스토리 관리

- 적절한 히스토리 크기로 메모리 사용량 제어
- 최신 네트워크 조건 반영을 위한 오래된 데이터 제거

4. 분산 고려

- 지연시간 분산을 고려한 타임아웃 조정
- 네트워크 불안정성에 대한 적응

5. 모니터링

- 타임아웃 통계 지속적 모니터링
- 성능 지표와 타임아웃 설정 간 상관관계 분석

타임아웃 관리 시스템은 Autobahn BFT의 성능 최적화에 핵심적인 역할을 하며, 네트워크 조건 변화에 동적으로 적응하여 불필요한 지연을 최소화합니다.